

Examen d'entraînement LEPL1110 — Éléments Finis

Claude code

Mai 2026

Durée : 3 heures. Aucun document autorisé. Calculatrice non autorisée.

Les exercices sont indépendants et peuvent être traités dans n'importe quel ordre.

Exercice 1 — Équation de Poisson et éléments finis P_2

On considère le problème de Poisson en dimension 1 :

$$-u''(x) = f(x), \quad x \in (0, 1),$$

avec les **conditions aux limites de Dirichlet homogènes** :

$$u(0) = 0, \quad u(1) = 0.$$

1.1 — Formulation faible

Soit $V = H_0^1(0, 1) = \{v \in H^1(0, 1) : v(0) = v(1) = 0\}$.

Multiplier l'équation par une fonction test $v \in V$ et intégrer sur $(0, 1)$. En utilisant une intégration par parties, montrer que la formulation variationnelle s'écrit :

Trouver $u \in V$ tel que $a(u, v) = \ell(v) \quad \forall v \in V$,

où $a(u, v) = \int_0^1 u'v' \, dx$ et $\ell(v) = \int_0^1 f v \, dx$.

Expliquer pourquoi les conditions de Dirichlet sont dites **essentielles** (et en quoi elles diffèrent des conditions de Neumann).

1.2 — Éléments P_2 de Lagrange

On considère l'élément de référence $\hat{K} = [-1, 1]$ avec trois nœuds en $\xi_1 = -1$, $\xi_2 = 0$, $\xi_3 = 1$.

(a) Donner les **trois fonctions de forme** $N_1(\xi)$, $N_2(\xi)$, $N_3(\xi)$ sur $\hat{K}$.

Rappel : N_i est l'unique polynôme de degré 2 vérifiant $N_i(\xi_j) = \delta_{ij}$.

(b) Calculer les dérivées $N'_1(\xi)$, $N'_2(\xi)$, $N'_3(\xi)$ et vérifier la **partition de l'unité** :

$$N_1(\xi) + N_2(\xi) + N_3(\xi) = 1 \quad \forall \xi \in [-1, 1].$$

1.3 — Matrice de rigidité élémentaire P_2

On considère un élément physique $K_e = [x_e, x_{e+1}]$ de longueur h , avec $x(\xi) = x_e + \frac{h}{2}(1+\xi)$.

(a) Montrer que :

$$K_{ij}^{(e)} = \frac{2}{h} \int_{-1}^1 N'_i(\xi) N'_j(\xi) d\xi.$$

(b) Calculer les entrées $K_{11}^{(e)}$ et $K_{12}^{(e)}$.

(c) En déduire :

$$K^{(e)} = \frac{1}{3h} \begin{pmatrix} 7 & -8 & 1 \\ -8 & 16 & -8 \\ 1 & -8 & 7 \end{pmatrix}.$$

Vérifier que $K^{(e)} \mathbf{1} = \mathbf{0}$ et donner une interprétation physique.

1.4 — Application : $N = 1$ élément, terme source $f = 2$

On choisit $N = 1$ élément P_2 sur $[0, 1]$ ($h = 1$), avec les nœuds $x_0 = 0$, $x_1 = \frac{1}{2}$, $x_2 = 1$. On suppose $f(x) = 2$.

(a) Exprimer les fonctions de forme physiques $N_i(x)$ avec $\xi = 2x - 1$. Calculer $\mathbf{F}$ de composantes $F_i = \int_0^1 2 N_i(x) \, dx$.

(b) Après imposition des conditions de Dirichlet $u_0 = u_2 = 0$, résoudre le système réduit et calculer u_1 .

(c) Comparer u_1 à la valeur exacte $u(\frac{1}{2})$ de $u(x) = x(1 - x)$. Justifier.

1.5 — Lemme de Céa et convergence

(a) Énoncer le **lemme de Céa** pour un problème variationnel coercif et continu.

(b) L'erreur d'interpolation pour P_k satisfait $\|u - \pi_h u\|_{H^1} \leq Ch^k |u|_{H^{k+1}}$. Dédurre la vitesse de convergence de $\|u - u_h\|_{H^1}$ pour P_1 et P_2 .

(c) On suppose $|u|_{H^3} = 1$. Pour quelle valeur minimale de N l'erreur P_2 est-elle inférieure à 10^{-4} ? ($C = 1$.)

Exercice 2 — Le jeu des erreurs

Le code ci-dessous assemble la matrice de rigidité pour $-u'' = 0$ sur $(0, 1)$ en éléments P_2 uniformes, avec quadrature de Gauss-Legendre à 3 points.

Ce code contient volontairement un maximum de sept erreurs.

```
1 import numpy as np
2
3 def assemble_stiffness_p2(N):
4     """Matrice de rigidite P2 sur (0,1), N elements uniformes, sans
5         CLs."""
6     n_nodes = 2 * N + 1
7     h = 1.0 / N
8
9     x = np.linspace(0.0, 1.0, N + 1)           # a annoter si errone
10
11     K = np.zeros((n_nodes, n_nodes))
12
13     xi_q = [-np.sqrt(3.0 / 5.0), 0.0, np.sqrt(3.0 / 5.0)]
14     w_q = [5.0 / 9.0, 8.0 / 9.0, 5.0 / 9.0]
15
16     for e in range(N):
17         dof = [2 * e, 2 * e + 1, 2 * e + 1]    # a annoter si errone
18         Jac = h                                  # a annoter si
19                                         errone
20
21         for q in range(3):
22             xi = xi_q[q]
```

```

21     w  = w_q[q]
22
23     dN_dxi = [xi - 0.5,
24               2.0 * xi,           # a annoter si errone
25               xi + 0.5]
26
27     dxi_dx = h / 2.0             # a annoter si errone
28
29     dN_dx = [dN_dxi[k] * dxi_dx for k in range(3)]
30
31     for i in range(3):
32         I = dof[i]
33         for j in range(3):
34             J = dof[j]           # a annoter si errone
35             K[I, J] += dN_dx[i] * dN_dx[j] * w * Jac
36
37     return x, K                 # a annoter si errone

```

Liste des erreurs (à compléter) :

- ① _____
- ② _____
- ③ _____
- ④ _____
- ⑤ _____
- ⑥ _____
- ⑦ _____